JISTBA tool overview

Michal Gregor - Luke (Luonnonvarakeskus)

June 2026

1 Introduction

In this report we comment on the annotation process of ground truth video data. We provide an overview of the workings and functionality of a new annotation tool JISTBA (Joint Instance Segmentation, Tracking and Behaviour Annotation), that can be used to annotate ground truth Video Object Segmentation (VOS) data, i.e. having unique masks with their IDs covering individual animals throughout a video, or to annotate this VOS data together with behaviour data, where we have for each animal and each time behaviour labels describing what is the behaviour of that particular animal at that time.

It was absolutely essential for us to annotate some ground truth data for our tracking projects, so we can evaluate and finetune models on it. We surely don't want to manually annotate (paint the masks) for every frame. Hence we looked into some available computer vision semi-automatic annotation tools, but we weren't able to find any useful ones that we could directly use for VOS segmenting in a video. They are mostly focused on AI-assisted annotation in tasks like image classification, object detection, semantic and instance segmentation in images, or tracking of objects in a video with bounding boxes, but we weren't able to find any tool directly for VOS annotation.

In the end we decided to implement our own annotation tool. It is being released as an open-source project and will be further improved and used at least internally within LUKE for a following project to annotate data for cow behavior analysis. The source code and documentation can be found at: `https://github.com/Eruveoo/JISTBA`. We tried to make the documentation and installation guide as clear as possible, although it's possible that you will encounter some new errors while trying to make it work. Please contact: michalgregor39@gmail.com with any questions.

2 System Architecture

The tool is a combination of backend and frontend parts. The backend is a project in Python with all the core functionality, running the ML models and processing the video data, accessible via REST API endpoints. It is meant to be run on a cluster with access to GPUs. The frontend in React is then run

locally on the annotator's local machine, and they communicate together via a traditional REST API interface. The local machine and server need to be connected via an SSH tunnel.

The tool was developed with the help of an AI-assisted programming tool, Cursor. The frontend was almost purely "vibe coded" (generated using only prompts to an LLM, almost without looking at the code at all), but that wasn't really possible anymore for the backend module, which required much more attention and understanding of the code.

3 Model Setup

In our project, the motivation for creating a tool like this was as follows: we have videos of cows and we would like to have their instance segmentation, having masks covering the individual cows in every frame, each with its unique ID corresponding identically to one cow. We have available a very good image instance segmentation model, SAM3. Potentially, we could segment every frame of the video with it and then match the masks in adjacent frames via similarity, manually correcting errors afterward. However, that probably wouldn't be such a good approach, as this matching would most likely produce quite a lot of errors. It would also be incredibly inefficient to run the heavyweight SAM3 on every frame.

Hence the idea is to connect two different models: first the SAM3 model for initialization in the first frame, detecting the different cows, and then use some of the available VOS models which take SAM's detected masks as input and propagate them throughout the video. However, the models will make errors, and for that we need the human annotator in the loop to correct those. The tool was inspired by the idea in Track Anything, where they exactly combine the SAM model for detection and XMem for VOS. We now go through the app design explaining the annotator workflow.

4 App Design and Annotator Workflow

Figure 1 shows the initial page of the annotation tool. In the top right corner, there is a status indicating the availability of the backend. Underneath that is a selection of the 'Annotation Mode', either 'Standard VOS' or 'Behaviour annotation'. Below that is a "Continue a session" field, where the user can pass the identifier of some previous session and continue the annotation where they left off. And finally, there is the box to upload the video to be annotated. It opens a file manager after clicking on it, or the user can also just drag and drop the video into it.

It then uploads the video to the backend and extracts the individual frames. Once done, we get to the situation in Figure 2, where we see some basic descriptive information about the video like resolution and FPS in the right panel. We have downsampled our videos to have 854 width, while keeping the aspect

ratio, and to 10 FPS, which seems good enough and is more feasible for the models. Below it, there is a field for the SAM text prompt instructing SAM which objects to segment, for us just “cow”.

4.1 Standard VOS

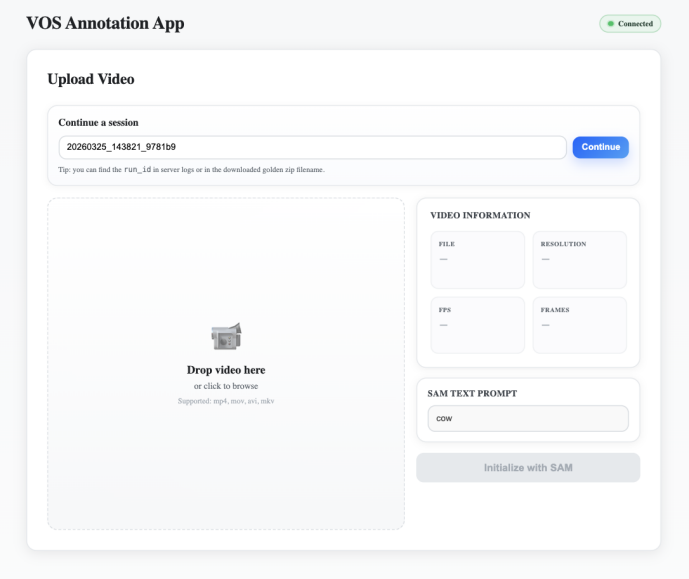


Figure 1: Initial page of the annotation tool.

When selecting ’Standard VOS’ as the annotation mode, after clicking “Initialize with SAM”, the SAM3 model processes the initial frame, detects the individual cows, and we get to the page in Figure 3. Here we see the initial frame of the video with the proposed SAM’s masks and IDs. The user can reassign the IDs and delete some masks, for example, of some cows in the background like cow 2 in this case.

After clicking “Apply IDs” we get to the main annotation section as visualized in Figure 4. On the right side, there is the “Golden Preview”, where we see the preview of the already annotated/correct segment of the video. In the beginning, it is just the initial frame. On the left side, there is then the preview of the currently tracked new segment to be validated by the annotator (empty from start). At the top, we see a progress bar showing the progress of how many frames from the video we have already annotated.

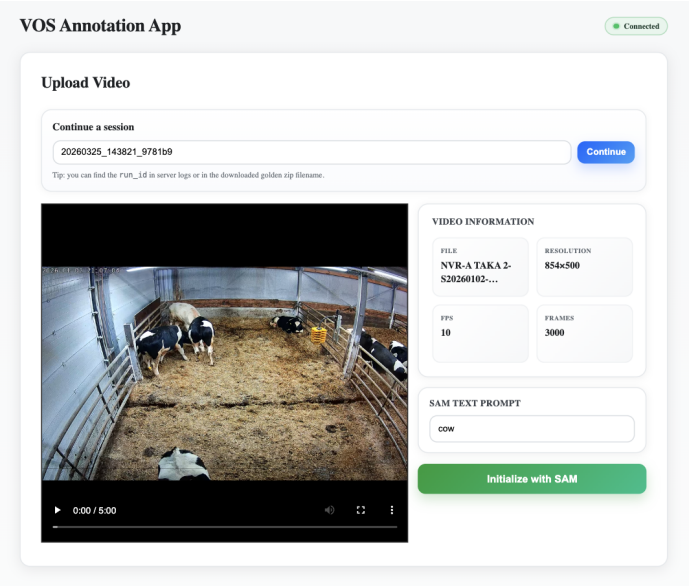


Figure 2: Interface after video upload.

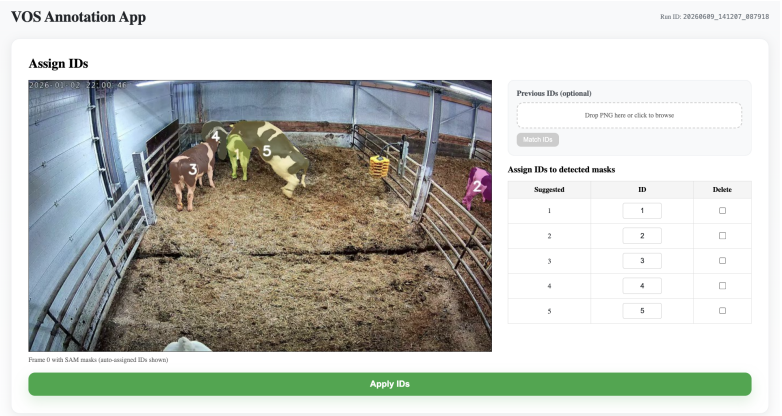


Figure 3: Interface after SAM initialization.

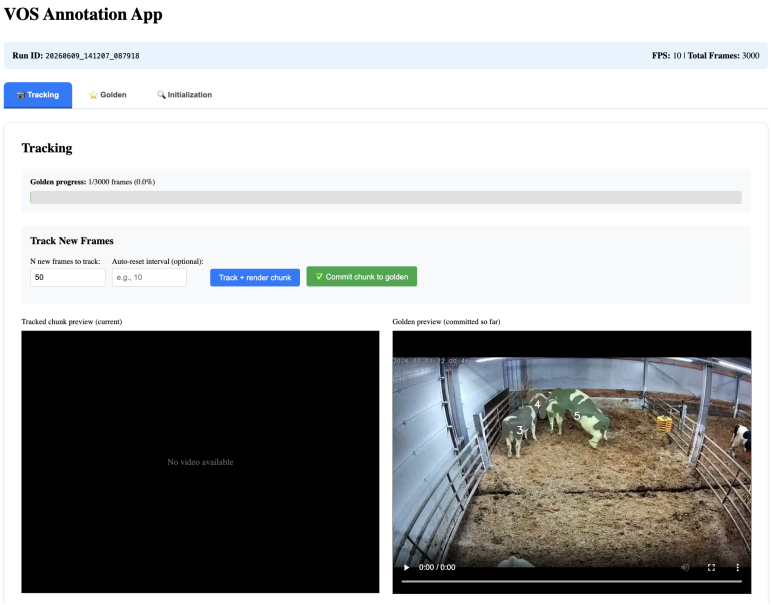


Figure 4: Main annotation interface with the Golden Preview and controls.

Underneath are two input fields and two buttons to be used during the annotation process. The first field "N new frames to track" should be filled with the number of how many new frames should be tracked with the XMem model, taking the last frame from the golden preview as the initialization. The second field "Auto-reset interval (optional)" is an optional field controlling the reinitialization feature, i.e., if we fill it with the number n, then after each n new frames tracked using XMem, the masks will be reinitialized using the SAM model and automatically re-identified with the masks taken from the last frame of the chunk previously tracked by XMem. This way we improve the quality of the generated masks as the masks generated by XMem tend to somewhat degrade over time.

"Track + render chunk" is the button that invokes the XMem model to track the selected number of new frames and shows the preview, and "Commit chunk to golden" is the button which moves this recently tracked video chunk to the golden data, if the annotator evaluates that it is correct. After pressing it, both the golden preview and the progress bar update as seen in Figure 5.

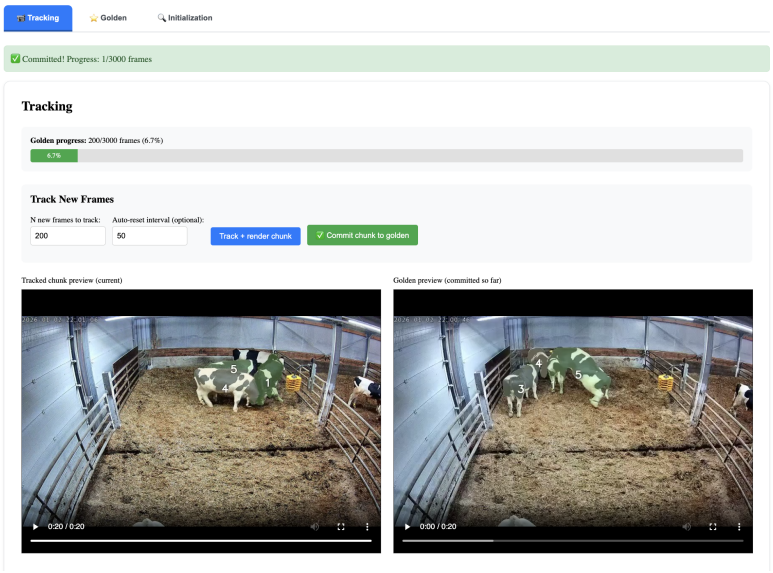


Figure 5: Interface after committing a correctly tracked chunk to golden data.

But it is of course possible that there is an error, which needs to be corrected. In that case, it is necessary to scroll a bit down to a correction section which is illustrated in Figure 6. Here we first need to identify the moment when the error appeared. There is again the preview of the currently tracked video chunk. By moving within the video, the annotator identifies the moment that the error appeared, confirms it by pressing "Select current frame", and presses "Prepare correction (run SAM)". This does two things. First, it copies all the frames from the recently tracked chunk to the golden video, as those were correct, and it runs the SAM model on the current frame to be corrected.

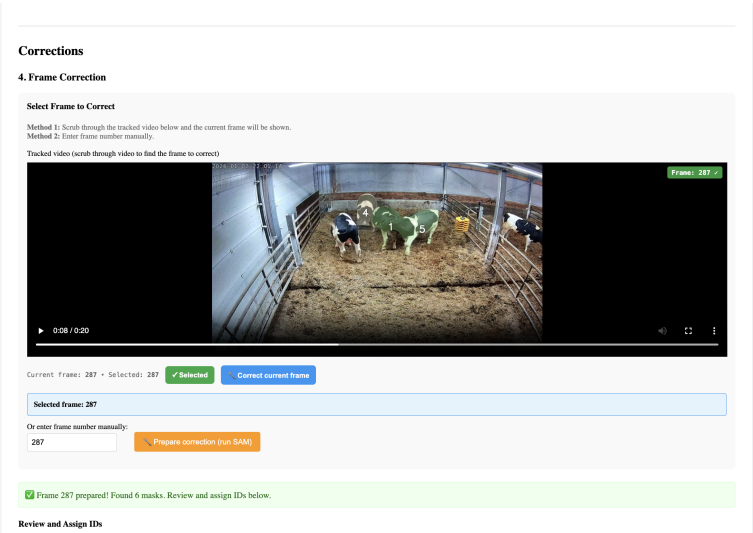


Figure 6: Correction section for identifying tracking errors in the video.

In Figure 7, we can see then what this SAM's prediction looks like. It tries to detect all the cows in the scene, trying to correctly assign IDs based on similarity. There is a table of the different detected cows on the right side, with their assigned IDs. It is again possible to reassign those IDs or to delete some, just like during the first initialization. It may, for example, be necessary again to delete the detected cows in the background.

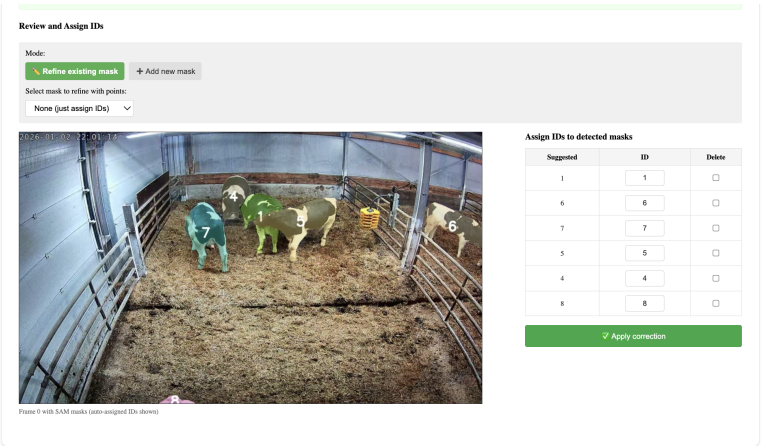


Figure 7: SAM prediction interface during the correction process.

But it is possible that this SAM's detection actually is not fully correct and needs further correction. It can for example happen that it doesn't detect one

cow as artificially illustrated in Figure 8. In that case, there is an "Add new mask" button, which allows the user to force the model to generate a new mask using a positive point prompt (by clicking at a point telling the model "There should be a cow here"). The result of doing so is then visible in Figure 9.

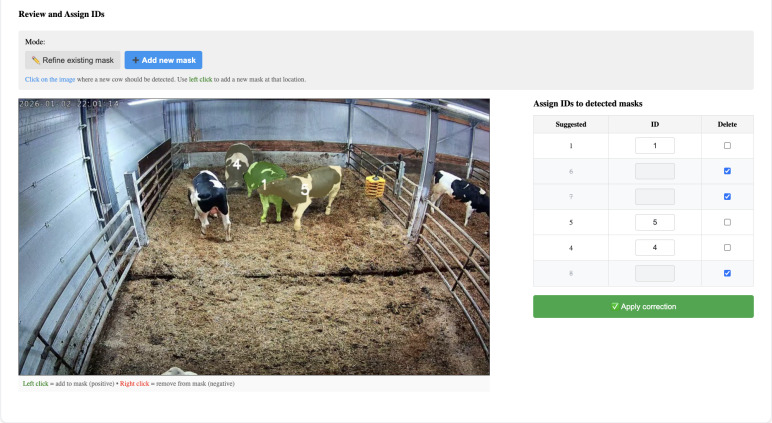


Figure 8: Illustration of an undetected cow requiring manual intervention.

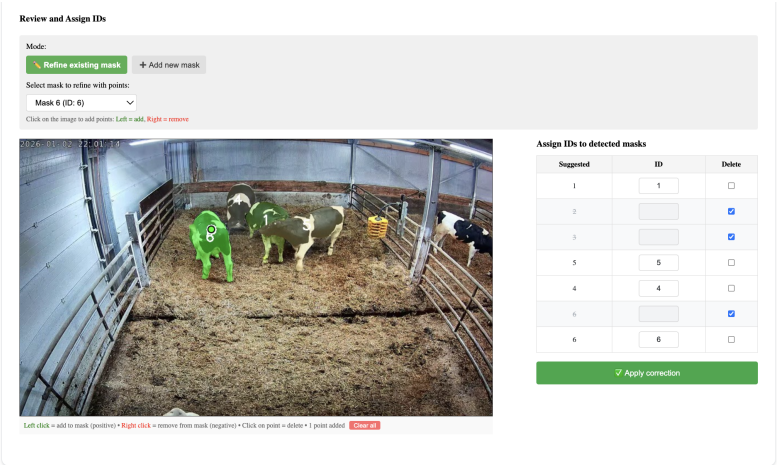


Figure 9: Result of adding a new mask using a positive point prompt.

Similarly, it is also possible to just adjust an existing mask using the "Refine existing mask" button, which allows the user to select the mask to be refined and correct it using both positive (left click) and negative (right click) point prompts. After all corrections are done, the user presses "Apply correction", which moves this corrected frame to the golden video and the annotator can further continue with the annotation by repeating the previous steps.

After the whole video is processed, the annotator can move to the “Golden” section from the top bar illustrated in Figure 10, which just shows again the preview of the annotated video for a final check. When done, the resulting annotated ground truth mask data can be downloaded by pressing the “Download golden folder” button.

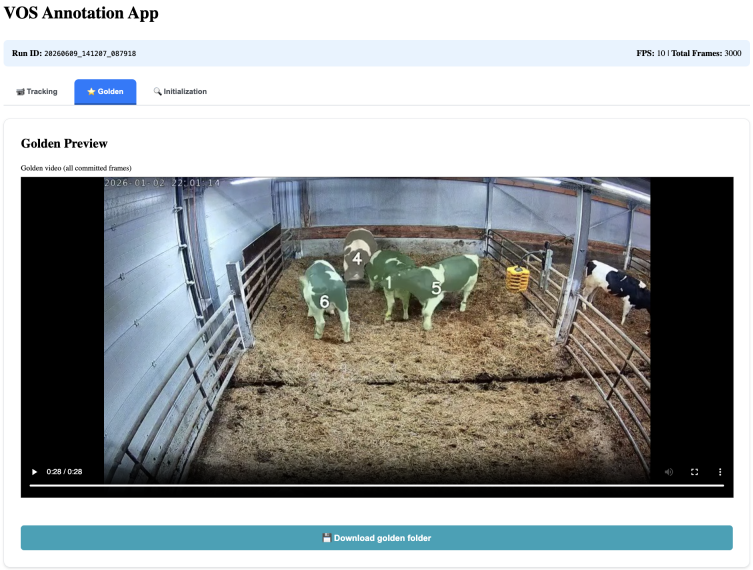


Figure 10: The Golden section for final review and data export.

4.2 Behaviour annotation

We now describe the behaviour annotation pipeline. If the user selects ’Behaviour annotation’ as the Annotation Mode, uploads a video, and presses ’Initialize with SAM’, they are directed to the page shown in Figure 11. Similar to the ’Standard VOS’ mode, one can edit the initial detection using SAM3 by modifying the IDs of the detected animals and deleting unwanted background detections. Additionally, the annotator selects the behaviour labels for this initial frame, describing what the animals are doing. There are three independent labels describing the behaviour of the animals. Label 1 is expected to always be non-trivial, whereas Labels 2 and 3 can be set to ’none’. The possible values for the behaviour labels can be customized for different animals, as described in the project’s README.

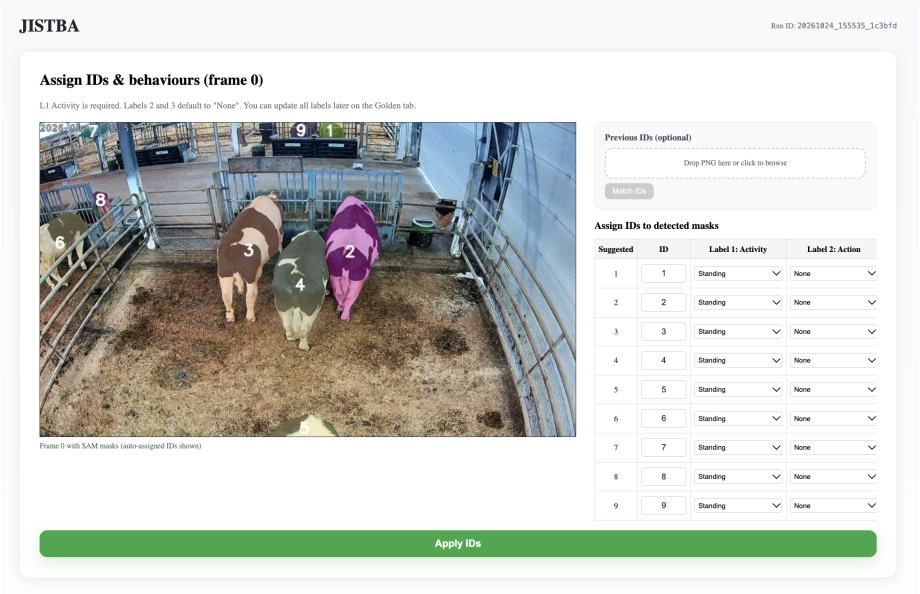


Figure 11: Initial detection interface using SAM3 for ID assignment and initial behavior label selection.

After assigning the IDs, setting the initial behavior labels, and pressing 'Apply IDs', the user proceeds to the main annotation interface (which is very similar to the Standard VOS mode). The Tracking section is exactly the same. Here, the annotator performs the exact same VOS annotation work as in the Purely VOS mode, iteratively running the XMem tracking model and correcting any mistakes using the SAM3 model.

However, the 'Golden' section is different, as this is where the behavior annotation takes place. First, there is a clean source video to watch. This is useful because the masks and behavior labels in the annotated preview can obscure fine behavioral details, making the clean video a helpful reference for the annotator. Underneath that is the annotated preview, which shows the video with masks covering the individual animals along with their behavior labels. These are displayed in Figure 12.

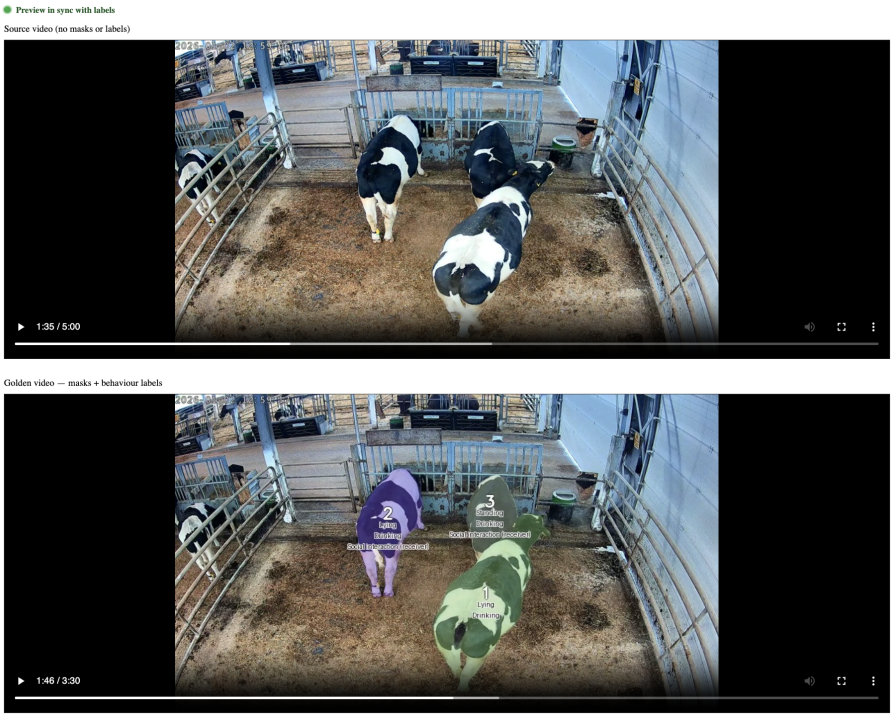


Figure 12: The Golden section displaying the clean source video and the annotated preview with masks and behavior labels.

Below the preview is the Behavior timeline, as shown in Figure 13. This serves as the main tool for behavior annotation. There is a row for each individual animal, which is further divided into three sub-rows for the different behavior labels. Each behavior is displayed as a colored section on this timeline, indicating a start and end time. The annotator can click on a particular time point in the video timeline, moving the indicator on the behavior timeline to add a behavior label. Vice versa, moving the selected point in the behavior timeline updates the preview video accordingly, allowing the annotator to see which frame in the video corresponds to that specific time point. The workflow with this behavior timeline is quite intuitive and fast. After making changes to the behaviour annotation using the timeline, the annotator can press the 'Rebuild golden preview' button to update the preview video with the new labels. This is not done automatically after every change, as the rendering process takes some time.

Once the annotation work is complete, the user navigates to the bottom of the page and presses 'Download golden folder' to download the annotated ground truth data from the session. The downloaded ground truth folder contains the video preview (with annotated masks and behavior labels), an An-

notations folder with the masks, and a JPEGImages folder. Similar to the 'Standard VOS' case, it follows the traditional VOS structure: one folder for the video frames and another for the annotated masks saved as PNG images for each frame. Additionally, there are three JSON files containing the behavior annotations, one for each label. These files contain clearly structured behavior annotations for each animal, indicating which activity is performed by which individual from start frame to end frame. This data can be used for further behavior analysis of the animals using statistical or machine learning techniques.



Figure 13: The Behavior timeline interface for interactive annotation of animal behaviors over time.